

Deep Learning for NLP

Student name: Μπαρμπάρσος Θεοφάνης
sdi: sdi2200107

Course: *Artificial Intelligence II (M138, M226, M262, M325)*
Semester: *Fall Semester 2023*

Contents

1	Abstract	2
2	Data processing and analysis	2
2.1	Pre-processing	2
2.2	Data partitioning for train, test and validation	3
2.3	Vectorization	3
3	Algorithms and Experiments	3
3.1	Experiments	3
3.1.1	Table of trials	4
3.2	Hyper-parameter tuning & Optimization techniques	4
3.3	Evaluation	5
3.3.1	Learning Curve	5
3.3.2	Confusion Matrix	6
4	Results and Overall Analysis	7
4.1	Results Analysis	7
4.1.1	Best Trial	8
4.2	Comparison with the First Project	8
4.3	Comparison with the second project	9
4.4	Comparison with the third project	9
5	Bibliography	9

1. Abstract

In this project, we developed a sentiment analysis classifier using deep neural networks, specifically leveraging PyTorch and pre-trained Word2Vec embeddings, to classify the sentiment of English-language tweets as either positive or negative. Building upon the dataset from the previous assignment, this work explores the use of word vector representations in conjunction with feedforward neural network architectures to enhance classification performance. The project involved careful design choices around model architecture, regularization techniques, training algorithms, and hyperparameter tuning to ensure robust and generalizable results. The complete pipeline included data preprocessing, embedding integration, model training, neural network fine-tuning, and evaluation using appropriate performance metrics to assess the effectiveness of the deep learning approach.

2. Data processing and analysis

2.1. Pre-processing

In this step, I implemented multiple preprocessing techniques to clean and normalize the Twitter dataset, ensuring it was suitable for feature extraction through tokenization-vectorization and classification. The following methods were applied:

1. **Lowercasing:** All text was converted to lowercase to ensure consistency and eliminate redundancy in token representation.
2. **Replacing Emails, URLs, and Mentions:** Using regular expressions, email addresses were replaced with the token "email", URLs with "url", and Twitter mentions (e.g., @username) with "username", as they do not contribute significantly to sentiment analysis.
3. **Handling Hashtags:** Hashtags were processed by removing the "#" symbol while preserving the associated word (e.g., "#happy" becomes "happy").
4. **HTML Entity Decoding:** HTML-encoded characters were converted to their corresponding textual representation (e.g., "&" becomes "&").
5. **Handling Repeated Characters:** Words with unnecessarily long character repetitions were normalized by limiting repetitions to at most two occurrences (e.g., "soooo" becomes "soo").
6. **Expanding Contractions:** Common English contractions were expanded to their full forms (e.g., "haven't" becomes "have not") to enhance textual clarity and standardization.
7. **Removing Non-Alphanumeric Characters:** Any character that is not an English letter, number, or whitespace was removed to eliminate noise and improve text quality.
8. **Spell Checking:** A spell checker (e.g., PySpellChecker) was applied to correct misspelled words, ensuring more accurate text representation.

2.2. Data partitioning for train, test and validation

In order to train, validate and test the model three datasets were given: train_dataset, val_dataset, test_dataset. In my implementation, the train_dataset was used to define the hyperparameters and the model's architecture, whereas the validation dataset was only used for the last training and evaluation. After this evaluation, the final model is used on the test dataset to output a submission file for the competition on Kaggle. The three datasets undergo the same pre-processing that includes all the techniques mentioned above along with tokenization and vectorization with the aid of Google's Word2Vec dataset for vectorizing tokens(it is used as input for this project and can be found under the "input" category).

2.3. Vectorization

As mentioned above, all three datasets go through the same process of pre-processing and tokenization before we apply the vectorization function on them. In essence, this function uses the Word2Vec dataset as the word-embedding of my choice. This contains the mapping of many words on their respective vectors. This way, for each token of every sentence the functions searches for a vector representation and if there is one, the token is replaced with it. If none is found, a zero vector replaces the token. This choice ensures that tokens that happen to not appear in the dataset will play no role in the whole sentence's vectorization and therefore, sentiment. Once this is done, every sentence(previously a number of words-tokens) is now a series of vectors, each representing one of the tokens. In order to calculate the vector for a given sentence, the mean value was used. Ultimately, every tweet, entry in the dataset, sentence, is represented as a single vector, this is the mean value of the vector representing each token of the sentence. The mean value was the optimal choice that could be made, without changing the approach of model creation too much.

3. Algorithms and Experiments

3.1. Experiments

To tackle the sentiment analysis problem, I started by experimenting with various pre-processing techniques to clean up the text data before training the model. Before any "real" work is done, i use a random seed to ensure reproducibility in my model creation. Initially, I applied basic text-cleaning methods such as lowercasing and removing non-alphanumeric characters. However, I progressively refined my approach by testing different techniques and analyzing their impact on the model. Also, i experimented with different neural network architectures that included 3, 4 and 5 layers, different activation functions, optimizer and many more(discussed in detail below). The trials below have an accuracy score alongside them to showcase relative results, in comparison to other trials. The numbers mentioned refer to local trials on my machine and on the validation dataset, therefore they may not be indicative of the score achieved on Kaggle. Despite that, they provide enough information to compare trials and fine-tune the model.

3.1.1. Table of trials. At first, the neural network consisted of four layers: the final is a sigmoid and all 3 remaining were linear, with no activation functions, Adam as the optimizer and BCE as a loss function. The loss function was BCE throughout the project because it is suitable for binary classification and BCEWithLogits would be redundant since I am already using a sigmoid function in the last layer of the network. Learning rate was set to 0.0001, dropout_rate to 0.2, weight decay to 3.5 and the layers size as:

D_in: x.shape[1] (size of the input sample)

D_out: 1

H1 = 256

H2 = 128

H3 = 64

Trial	Score
Only lowercasing for pre-processing	0.7553217840
All the techniques without pyspellchecker	0.7689041532
Full pre-processing	0.7926714389

Table 1: Trials

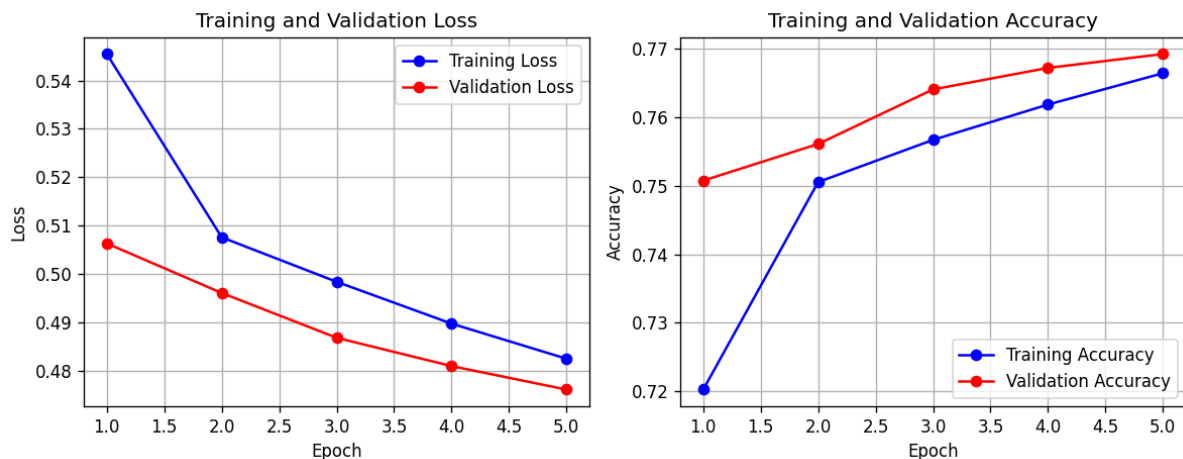
Afterwards, I experimented with 3 and 5 layers separately with exactly the same structure mentioned above, only with a layer less/extra. Trials produced, for 3 layers accuracy **0.7624803191** and for 5 layers accuracy **0.7742159084**. Based on the results, a 4-layer architecture was selected for the neural network and further experiments with other parameters followed.

3.2. Hyper-parameter tuning & Optimization techniques

In experimenting and fine-tuning the network's attributes and hyper-parameters, Optuna was used in order to try and find the best possible parameters while helping it choose between the best possible options as discussed in the course's lectures. Optuna was used to find the best value for each of the following:

- Size of layer 1 (H1): Chosen between 64 and 256 with a step of 32. Layer 2 (H2) is always half of H1, and Layer 3 (H3) is always half of H2.
- Learning rate: Sampled from a log-uniform distribution between 0.00001 and 0.01.
- Batch size: Chosen from the values {32, 64, 128}.
- Dropout rate: Sampled from a uniform distribution between 0.1 and 0.5.
- Weight decay: Sampled from a log-uniform distribution between 0.000001 and 0.01.
- Optimizer: Chosen from {SGD, Adam, AdamW}.
- Activation function: Chosen from {ReLU, LeakyReLU, Tanh}.

Figure 1:



For every trial, Optuna chooses one value for each of the above and trains the model for 5 epochs producing a loss value while also keeping the attributes and loss value of the trial with the minimum loss value yet. Optuna was ran for 200 trials and the results could be automatically parsed and used for the final training of the model but currently, there is no need for that. The trial process is commented out as well as the auto parsing of the best parameters, because the best parameters out of 200 trials were hardcoded exactly before the final training. As for over/underfitting, learning curves are displayed below (Figure 1) showcasing that the model's behavior is just as it should.

To mitigate overfitting and underfitting during training, several techniques were applied. First, dropout regularization was used with a tunable dropout rate to prevent the network from relying too heavily on specific neurons. Weight decay (L2 regularization) was also employed to penalize large weights and encourage simpler models. The model architecture and size of each layer were carefully tuned using hyperparameter optimization to ensure that the network had sufficient capacity without being overly complex.

3.3. Evaluation

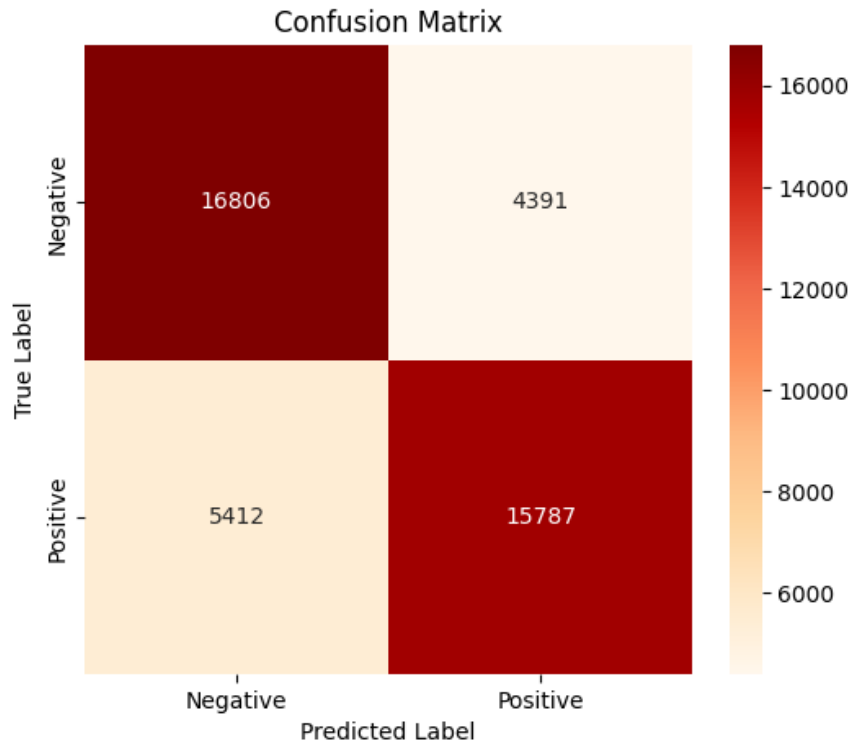
The model's performance was assessed using standard evaluation metrics:

- **Accuracy:** Measures overall correctness of predictions.
- **Precision, Recall, and F1-score:** Provides insights into false positives and false negatives.
- **Confusion Matrix:** Visualizes model misclassifications.

3.3.1. Learning Curve. The learning curve showcases training vs. validation accuracy over iterations. If there is a large gap between the two, it indicates overfitting. A properly tuned model should show convergence between these curves. (Figure 1)

3.3.2. Confusion Matrix. The confusion matrix (Figure 2) provides a breakdown of correct and incorrect predictions. It helps analyze where the model struggles (e.g., misclassifying positive sentiment as negative).

Figure 2:



The matrix is as follows:

$$\begin{bmatrix} TP & FN \\ FP & TN \end{bmatrix} \quad (1)$$

Where:

- **TP (True Positives):** Correctly predicted positive sentiments.
- **FN (False Negatives):** Positive sentiments misclassified as negative.
- **FP (False Positives):** Negative sentiments misclassified as positive.
- **TN (True Negatives):** Correctly predicted negative sentiments.

Analysis of these evaluation metrics, ensured model reliability and identified areas for further improvement.

4. Results and Overall Analysis

4.1. Results Analysis

Overall, the model achieved a validation accuracy of **0.7927** during local evaluation and a Kaggle test set accuracy of **0.77678**. While the results are not state-of-the-art, they reflect a well-generalized model considering the scope of the assignment.

The learning curves (Figure 1) confirm that the model is not overfitting — training and validation accuracy progress in sync, and validation loss consistently decreases.

This steady learning behavior implies that the architecture, regularization, and hyperparameter tuning were appropriate.

Nonetheless, further improvements could likely be achieved by extending the number of training epochs or performing additional hyperparameter optimization. Due to limited available time during the project phase, only 200 Optuna trials and 5 training epochs per trial were explored. With more time, experimenting with other embedding strategies, increasing training time, could yield better results.

4.1.1. Best Trial. The best trial configuration, as selected through Optuna, used the following parameters:

- Optimizer: **AdamW**
- Activation Function: **ReLU**
- Learning Rate: **0.00045107211839375217**
- Dropout Rate: **0.10020346423181177**
- Batch Size: **32**
- Hidden Layer Sizes: **H1 = 256, H2 = 128, H3 = 64**
- Weight Decay: **4.380515001378823e-06**

This configuration achieved the lowest validation loss and was chosen for the final training and evaluation. The effectiveness of dropout, regularization, and the activation function were crucial in preventing overfitting and ensuring a well-behaved model.

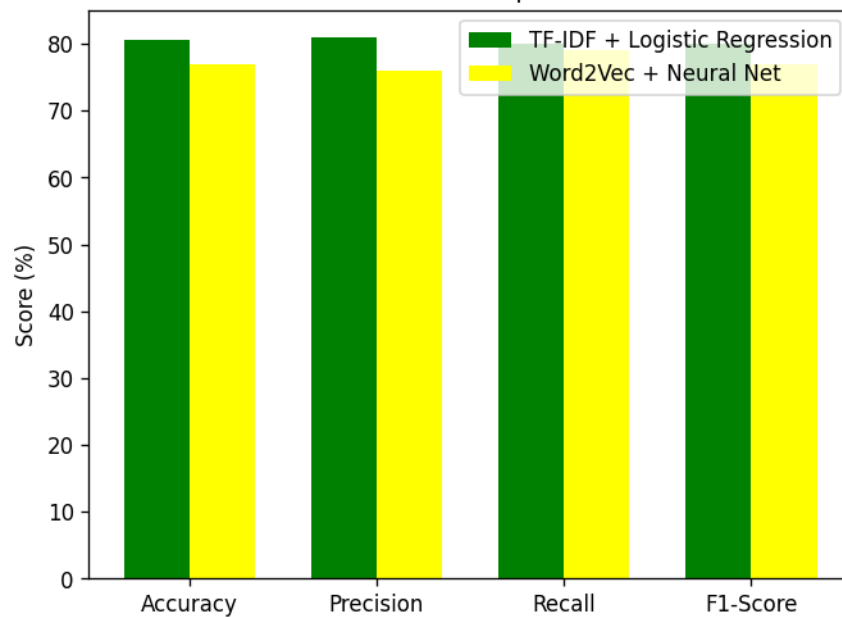
4.2. Comparison with the First Project

The first project employed a simpler machine learning approach using TF-IDF for feature extraction and Logistic Regression as the classifier. That model achieved a higher accuracy on both validation and test sets compared to the deep neural network used in this project. A visual comparison between evaluation of the two models from the two projects is displayed in Figure 3.

Despite the more advanced techniques used here (e.g., word embeddings, feedforward neural networks, and hyperparameter tuning), the lower performance may seem counterintuitive at first. However, it can be reasonably explained:

- Deep learning models often require more data and longer training to surpass traditional models in text classification tasks. Logistic Regression, when paired with TF-IDF, is highly effective for short, sparse text like tweets.
- Due to time constraints, we were not able to explore an extensive range of configurations or train the model over many epochs. This likely limited the neural network's ability to fully capitalize on its potential.
- Pre-trained Word2Vec embeddings, while rich, are not tailored to Twitter-specific language (slang, emojis, abbreviations), possibly weakening performance relative to TF-IDF's direct frequency-based representation.

Figure 3:
Models comparison



That said, this project lays a strong foundation for future enhancements. With more time and resources, integrating more modern embeddings (e.g., fine-tuned BERT) or deeper architectures could lead to significant improvements beyond traditional baselines.

4.3. Comparison with the second project

<Use only for projects 3,4>

<Comment the results. Why the results are better/worse/the same?>

4.4. Comparison with the third project

<Use only for project 4>

<Comment the results. Why the results are better/worse/the same?>

5. Bibliography

References

[1] Donald E. Knuth. Literate programming. *The Computer Journal*, 27(2):97–111, 1984.

<You should link and cite whatever you used or "got inspired" from the web. Use the / cite command and add the paper/website/tutorial in refs.bib>

<Example of citing a source is like this:> [1] <More about bibtex>